

PowerShell Cyber Defense Operations

Instructor Answer Key & Grading Guide (Part 2)



Section 1: Core Mechanics & Mid-Level Challenge Solutions

Step-by-Step Activity 11: Challenge Mode — Build the Master Investigative Chain!

Student Objective: Write a single pipeline command to isolate high-CPU processes, sort them from highest to lowest resource usage, and filter the data down to three precise columns (Process Name, Process ID, and CPU).

Expected Student Query:

```
Get-Process | Where-Object {$_.CPU -gt 10} | Sort-Object CPU -Descending | Select-Object ProcessName, Id, CPU
```

Grading Criteria — Cmdlet Consistency: Check that students are using full cmdlet names (Where-Object, Sort-Object, Select-Object) instead of standard short aliases (where, sort, select) as mandated by professional documentation rules.

Grading Criteria — Logical Order: The pipeline must flow from gathering data -> filtering rows -> sorting rows -> selecting columns. If Select-Object is placed before Sort-Object or Where-Object, the CPU or ID properties might be stripped prematurely, causing the pipeline to fail or return empty results.



Section 2: Final Boss Challenge — Operation Obscured Beacon

Phase A: Trace the Outbreak

Student Objective: Query the deep system infrastructure to identify how the rogue background notepad.exe instances are being spawned.

Expected Student Query:

```
Get-CimInstance Win32_Process | Where-Object Name -eq "notepad.exe" | Select Name, ProcessId, ParentProcessId
```

Expected Output Table Artifact:

Name	ProcessId	ParentProcessId
----	-----	-----
notepad.exe	4112	8420
notepad.exe	5304	8420
notepad.exe	6116	8420
notepad.exe	7892	8420

Key Diagnostic Insight: *The core objective of this phase is for the student to look at the unified ParentProcessId column (e.g., 8420) and realize that a single, hidden master script is responsible for generating the entire automated process outbreak. Exact numbers will vary dynamically based on system state, but all entries must match.*

Phase B: Expose and Audit the Master Process

Student Objective: Target the parent process ID uncovered in Phase A, explore its structural properties blindly using Get-Member to find the non-standard property field, and then extract the full five-column forensic metadata block.

Step 1: Property Discovery (The Hidden Sub-step)

```
Get-CimInstance Win32_Process | Where-Object ProcessId -eq 8420 | Get-Member
```

Investigation Discovery: Students will look through the resulting properties and notice the standard CommandLine field which reveals how the application was executed.

Step 2: Final Evidence Extraction Query:

```
Get-CimInstance Win32_Process | Where-Object ProcessId -eq 8420 | Select Name, ProcessId, ExecutablePath, CreationDate, CommandLine
```

Expected Evidence Table Output:

```
Name           : powershell.exe
ProcessId      : 8420
ExecutablePath : C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe
CreationDate   : 6/28/2026 3:41:34 AM
CommandLine    : powershell -WindowStyle Hidden -Command
"$Tag='Rm9yZXN0IGlzIENvb2tpbmcgT2JhaSBJZGVhIQ==' ; while($true){ Start-Process
notepad -WindowStyle Minimized; Start-Sleep -Seconds 8 }"
```

Question [3]: What was the name of the structural property field where the hidden tag was discovered?

Answer: **CommandLine**

Phase C: Bonus Activity — Decoding the Adversary

Student Objective: Identify the specific obfuscation profile used by the attacker and decode the scrambled text string found inside the \$Tag variable configuration block.

Deobfuscation Analysis:

- **Encoding Identification:** Students should recognize the character composition and distinct trailing double padding signs (==) as the definitive hallmark signature of Base64 Encoding.
- **Decoded String Payload:** Inputting the cipher string Rm9yZXN0IGlzIENvb2tpbmcgT2JhaSBJZGVhIQ== into an online decoder (e.g., CyberChef) yields the plain English campaign signature:

```
Forest is Cooking Obai Idea!
```

Phase D: Plot Twist — Neutralize and Evict the Threat!

Student Objective: Write the containment scripts to safely terminate the running automated loop structure and purge all leftover background windows.

Acceptable Solution Option 1 (The Advanced CIM / WMI Method):

This approach demonstrates a high-level understanding of object pipelines by isolating the exact parent process via its execution string parameters before invoking termination commands:

```
# 1. Stop the simulated malware parent engine loop
Get-CimInstance Win32_Process | Where-Object { $_.Name -eq 'powershell.exe' -and
$_ .CommandLine -match 'Start-Process notepad' } | ForEach-Object { Stop-Process -
Id $_.ProcessId -Force }

# 2. Close out all remaining background Notepad windows
Get-Process notepad | Stop-Process
```

Acceptable Solution Option 2 (The Targeted Standard Process Method):

This alternative approach utilizes standard process maps to target the specific hidden instance of powershell by matching it against its distinctive blank title bar and clearing out the remaining processes:

```
# 1. Terminate the hidden background parent execution loop
Get-Process -Name powershell | Where-Object { $_.MainWindowTitle -eq "" } | Stop-
Process -Force

# 2. Purge the remnant orphan client windows completely
Get-Process -Name notepad | Stop-Process
```